

SE322: Software Design and Architecture

Activity 5: Principles of Detailed Design

Objective: this activity, the students will learn how to analyze an existing software architecture or design to check if the code violates principles or not.

Expected Outcome: After completing this exercise, the students will be able to:

1. Analyze code to check if the code violates principles or not
2. Analyze class diagram to check if the diagram violates principles or not

Course Learning Outcome:

1. CLO 1: Recognize the principles, strategies, and process of software design and architecture.
2. CLO 2: Analyze an existing software architecture or design.

Question1: Consider the code

```
public class Shape {
    ShapeType type;
}
public class Square extends Shape
{
    public Square() { this.type = ShapeType.SQUARE; }
    double side;
}
public class Circle extends Shape
{
    public Circle() { this.type = ShapeType.CIRCLE; }
    double radius;
    Point center;
}
public class Server
{
    void drawSquare(Square square)
    {
        System.out.println("I'm drawing a square");
    }

    void drawCircle(Circle circle)
    {
```

```
        System.out.println("I'm drawing a circle");
    }

    void drawAllShapes(List<Shape> shapes)
    {
        for (Shape shape : shapes)
        {
            switch (shape.type)
            {
                case SQUARE:
                    drawSquare((Square) shape);
                    break;
                case CIRCLE:
                    drawCircle((Circle) shape);
                    break;
            }
        }
    }
}
```

1. Does this code violate the OCP principle? Why?

Yes, the code violates the Open/Closed Principle (OCP).

The OCP states that software classes should be open for extension but closed for modification.

In the given code, the `Server` class uses a switch statement to determine which shape to draw.

Whenever a new shape is added, such as a `Triangle`, the `Server` class must be modified to support it.

This violates OCP because adding new functionality requires changing existing code.

2. If it is violated, rewrite the code in a way to comply with the OCP Principle.

We can solve this problem by using abstraction and polymorphism. Instead of having the `Server` class decide how to draw each shape, every shape will implement its own `draw()` method.

```
abstract class Shape {
    abstract void draw();
}

class Square extends Shape {
    double side;
```

```

    @Override
    void draw() {
        System.out.println("I'm drawing a square");
    }
}

class Circle extends Shape {
    double radius;
    Point center;

    @Override
    void draw() {
        System.out.println("I'm drawing a circle");
    }
}

class Server {
    void drawAllShapes(List<Shape> shapes) {
        for (Shape shape : shapes) {
            shape.draw();
        }
    }
}

```

Now, each shape is responsible for drawing itself. If we want to add a new shape, we only need to create another subclass of `Shape` and implement the `draw()` method.

The `Server` class does not need to be modified, so the design follows the Open/Closed Principle.

Question2: Banking Application example

Banking application supports two account types – “current” and “savings”. The bank now wants to offer a high interest-earning fixed-term deposit account to its customers.

1. Consider the class diagram does it violate the LSP principle if the bank create fixed-term deposit as new account? Why?

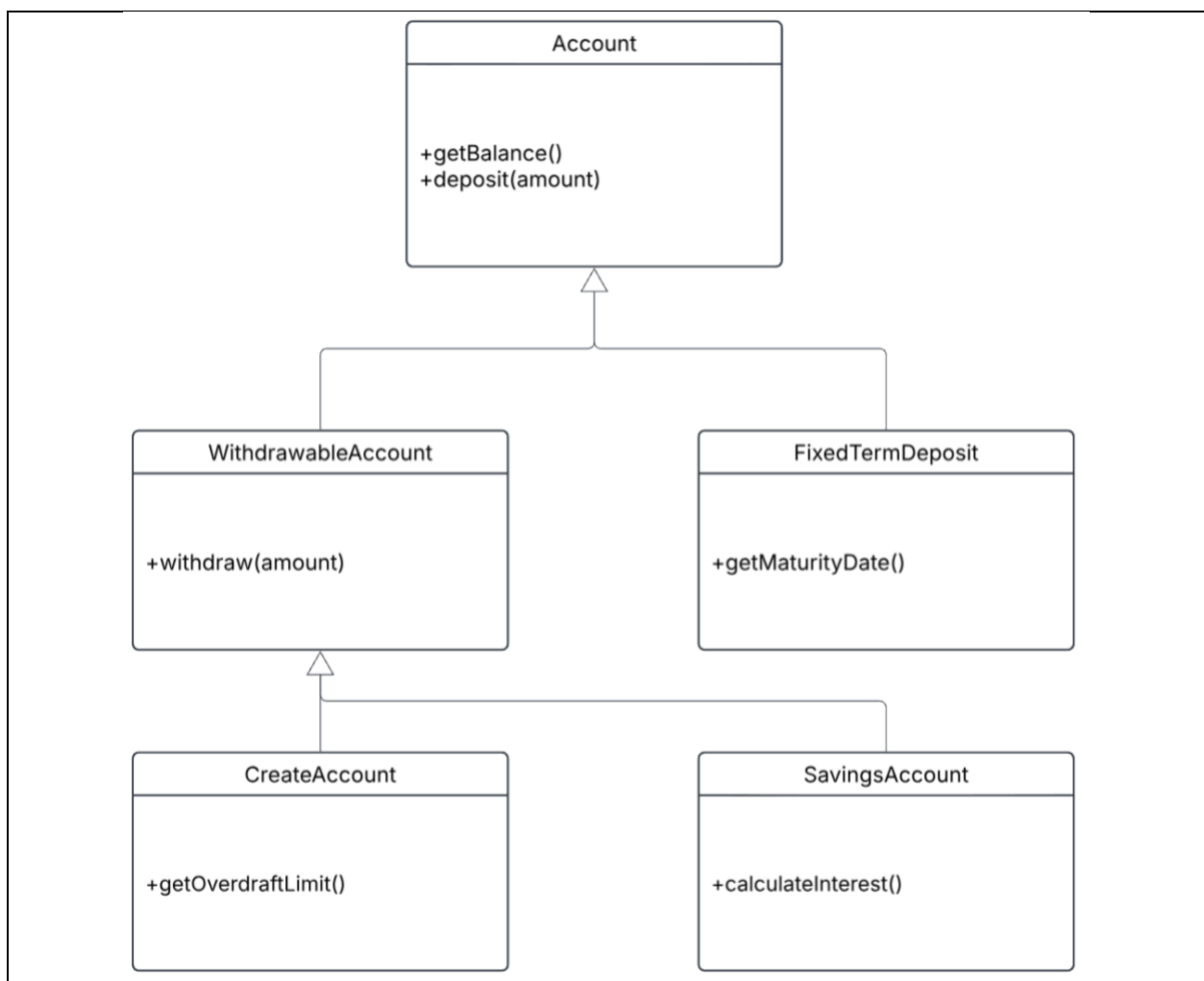
Yes, the design violates the Liskov Substitution Principle (LSP) if `FixedTermDepositAccount` inherits a `withdraw()` operation that it cannot support.

LSP states that subclasses should be able to replace their parent classes without affecting the correctness of the program.

In this case, current and savings accounts allow withdrawals, while a fixed-term deposit account generally restricts withdrawals until maturity.

If the fixed-term deposit account inherits a withdrawal method but cannot perform it, it cannot fully substitute for its parent class. Therefore, it violates LSP.

2. If it is violated, redraw the diagram in a way to comply with the LSP Principle.



The `Account` class contains operations shared by all accounts, such as `deposit()` and `getBalance()`.

The `WithdrawableAccount` class adds the `withdraw()` method and is inherited by `CurrentAccount` and `SavingsAccount`.

The `FixedTermDeposit` class inherits directly from `Account` because it does not support ordinary withdrawals before maturity.

This design follows LSP because subclasses are only required to support operations they can perform correctly.

